Week 6: From Random Forest to LightGBM

Mastering Gradient Boosting for Medical Data

MDST Project: Heart Failure Survival Analysis

2026

Learning Objectives

By the end of this week, you will:

- ✓ Understand **Random Forest baseline**
  (parallel independent trees from Week 5)
- ✓ Learn **Gradient Boosting concept**
  (sequential error correction)
- ✓ Master **LightGBM**
  (modern leaf-wise boosting with hyperparameter optimization)
- ✓ Evaluate using **4 metrics**
  (Accuracy, F1-Score, ROC-AUC, MCC)
- ✓ Get **hands-on practice**
  (hyperparameter tuning with Optuna)

Evaluation Metrics: Following Chicco & Jurman (2020)

Metric	Formula	Interpretation	Best For
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	% correct	Balanced
F1-Score	$2 \cdot \frac{P \cdot R}{P+R}$	Precision-Recall	Imbalanced
ROC-AUC	Area under curve	Ranking quality	Threshold-free
MCC	Correlation coeff.	Most robust	Binary imbalanced

Key Insight: No single metric tells the full story.

Report all four metrics for comprehensive evaluation

Why Multiple Metrics Matter: A Real Example

Scenario: Heart failure classification (10% positive class)

Model	Accuracy	F1	ROC-AUC	MCC
A: Always predict "No"	90%	0%	50%	0%
B: Balanced predictions	75%	60%	85%	50%

The Problem: Model A looks excellent by accuracy alone!

The Solution: F1, ROC-AUC, and MCC expose the failure.

Lesson

Accuracy alone is **misleading** on imbalanced data.
Use MCC and ROC-AUC as primary metrics.

Random Forest: Parallel Learning (Week 5 Review)

How It Works:

Bootstrap sampling (random rows with replacement)

$\downarrow$

Train trees independently on different samples

$\downarrow$

Each tree sees random subset of features

$\downarrow$

Majority vote on final prediction

Key Characteristics:

- **Process:** Parallel (all trees trained simultaneously)
- **Tree depth:** Deep (10-12 levels)
- **Learning mechanism:** Variance reduction via averaging
- **Speed:** Fast training
- **Role:** Good baseline for Week 5

Random Forest In Action: Voting Example

Example with 4 trees:

```
Original data: [row1, row2, row3, row4, row5]

Tree 1 (bootstrap sample): [row1, row1, row3, row5,
    row5]      Yes
Tree 2 (bootstrap sample): [row2, row3, row3, row4,
    row5]      No
Tree 3 (bootstrap sample): [row1, row2, row4, row4,
    row5]      Yes
Tree 4 (bootstrap sample): [row2, row3, row4, row5,
    row5]      Yes

Final Prediction: Yes  (3/4 trees agree = majority
    vote)
```

Why Averaging Works:

- Each tree is a **high-variance** estimate (overfits to its sample)
- Errors are **uncorrelated** across samples

Gradient Boosting: Sequential Learning Paradigm

The Sequential Idea:

Iteration 1: Train Tree_1 on original data

Iteration 2: Compute residuals (errors) from Tree_1
Train Tree_2 on these residuals

Iteration 3: Compute residuals from $\text{Tree}_1 + \text{Tree}_2$
Train Tree_3 on these residuals

. . . repeat . . .

Final Prediction: $\text{Tree}_1 + \alpha\ \text{Tree}_2 + \alpha\ \text{Tree}_3 + \ldots$

Each new tree focuses on fixing mistakes from previous trees

Why Gradient Boosting Outperforms Random Forest

Aspect	Random Forest	Gradient Boosting
Bias Reduction	No	Yes
Variance Reduction	Yes	Yes
Tree Depth	Deep (10-12)	Shallow (3-5)
Can Learn Errors	No	Yes
Typical Accuracy	∼80-85%	∼85-90%

Mathematical Insight:

- **Random Forest:** $\text{Error} = \text{Bias}^2 + \text{Variance}$
  - Reduces variance only
  - Asymptotically: $\text{Error} \approx \text{Bias}^2$ (bottleneck)
- **Gradient Boosting:** $\text{Error} = \text{Bias}^2 + \text{Variance}$
  - Reduces **both** bias and variance
  - Better asymptotic error

Gradient Boosting Example: Learning from Residuals

Realistic Scenario:

```
Data samples: [(age=50, died=No),
               (age=60, died=Yes),
               (age=55, died=No)]

Tree 1 predictions:  [0.0,  0.8,  0.2]
Actual labels:       [0.0,  1.0,  0.0]
Residuals:          [0.0, +0.2, -0.2]      Tree 2
   focuses here

Tree 2 learns:
  - For sample 2: Predict +0.2 (boost the prediction)
  - For sample 3: Predict -0.2 (reduce the prediction)

After Tree 1 + 0.1    Tree 2:
  Predictions: [0.0, 0.82, 0.18]

  Closer to actual [0.0, 1.0, 0.0]
```

Gradient Boosting Hyperparameters Explained

n_estimators (number of trees)

- More trees = more learning iterations
- Trade-off: More complexity, slower training
- Typical range: 50 to 200

learning_rate (step size, also called 'shrinkage')

- lr = 0.1: Each tree contributes 10% to prediction
- lr = 0.05: Each tree contributes 5% (slower, more stable)
- lr = 0.01: Each tree contributes 1% (very slow, careful learning)
- Rule of thumb: Lower lr needs more trees

max_depth (tree complexity)

- depth = 3: Shallow trees (avoid overfitting)
- depth = 5: Medium trees (balance)
- depth = 10: Deep trees (high variance, prone to overfitting)

subsample (data fraction per tree)

- 0.8 = use 80% of data for each tree
- Adds randomness, prevents overfitting

Gradient Boosting Training Curve Intuition

Learning trajectory on training data:

```
Iteration 1: Train on full data
  Misclassification rate: 25%

Iteration 2: Train on residuals of 25% samples
  New misclassification: 20%

Iteration 3: Train on residuals of remaining 20%
  New misclassification: 18%

Iteration 4: Train on residuals of remaining 18%
  New misclassification: 17%

Iteration 5: Train on residuals of remaining 17%
  New misclassification: 16.5%

(Diminishing returns as we improve)
```

Pattern: Learning curve is steep at first, then flattens

LightGBM: The Modern Upgrade

Traditional Gradient Boosting (Level-wise):
All nodes at depth d split before any node at depth $d + 1$.

```
        node
       /    \
     node    node      <- depth 1
    /  \    /   \
  node node node node   <- depth 2
  /\   /\   /\   /\
 l  l l  l l  l l  l    <- depth 3 (ALL split)
```

LightGBM (Leaf-wise):
Splits only the leaf that reduces loss most (smart focus).

```
        node
       /    \
     node    node
    /  \      |
  node node  node   <- splits only where it matters most
```

Result: Fewer trees needed, **10-20× faster**, often better accuracy.

LightGBM vs Other Boosting Algorithms

Algorithm	Growth	Speed	Accuracy	Overfit
GB (sklearn)	Level-wise	Baseline	Good	Low
XGBoost	Level-wise	Medium	Very Good	Medium
LightGBM	**Leaf-wise**	**Very Fast**	**Excellent**	**High***
CatBoost	Symmetric	Slow	Excellent	Medium

** High overfitting risk on small data; requires careful regularization*

For our dataset: Heart failure data has only 299 samples.

- LightGBM with strong regularization is optimal
- Conservative hyperparameters prevent overfitting
- Benefit: 10× speedup vs sklearn GB

LightGBM Hyperparameters

num_leaves (tree complexity limit)

- 15: Conservative (prevents overfitting)
- 31: Default, balanced
- 100: Aggressive (high variance, risk on small data)

learning_rate

- Same interpretation as GB
- Default: 0.1 (standard), 0.05 (slower, stable)

min_data_in_leaf

- Minimum samples required in each leaf
- Default: 20
- Heart failure data: Use 20-30 as safety valve against overfitting

feature_fraction & bagging_fraction

- Randomness in features and samples
- Default: 1.0 (use all)
- Recommended: 0.8 (use 80%)

max_depth

- Explicitly limit depth (safety mechanism)

Why Leaf-wise Growth is Smarter

The Smart Decision:

```
Training data analysis:
  Node A: 100 samples with 30% error rate
  Node B: 80 samples with 5% error rate

Level-wise (traditional):
  Split both A and B equally
      Wastes effort on node B

Leaf-wise (LightGBM):
  Split A first (30% error is higher!)
      Fix the bigger problem first
      Then split B (5% error is now bigger)

Result: Finds high-error regions faster
        with fewer total splits needed
```

The Danger:

Why Optuna for Hyperparameter Tuning?

Random Search:

- Try 50 random combinations
- Expected good params: Maybe 2-3 by luck
- Wasteful

Grid Search:

- Try $3 \times 3 \times 3 \times 3 = 81$ combinations
- Computational cost: Massive
- Covers only discrete points

Bayesian Optimization (Optuna):

- Try 50 trials, but **smart**
- Trials 1-10: Explore broadly
- Trials 11-30: Focus on promising regions
- Trials 31-50: Refine around the best
- Expected good params: 10-20

Optuna wins: Best quality with fixed computational budget.

Optuna Strategy for Our Model

The optimization function:

```
def objective(trial):
    # Suggest hyperparameters
    n_est = trial.suggest_int('n_estimators', 50, 150)
    lr = trial.suggest_float('learning_rate', 0.01,
        0.1, log=True)
    num_leaves = trial.suggest_int('num_leaves', 15,
        50)
    # ... more hyperparameters ...

    # Train and evaluate
    model = LGBMClassifier(**params)
    scores = cross_val_score(model, X, y, cv=5,
                             scoring='roc_auc')

    # Return single metric for optimization
    return scores.mean()

# Let Optuna run 50 trials
```

The Complete ML Pipeline

Raw Data (299 samples, 11 features)

↓

Train/Test Split (70/30) + StandardScaler

↓

Training Set (210) →
Test Set (89)
(use only for training)
(final evaluation)

Training Set

↓

Section 2: Random Forest (baseline)
Section 4: Gradient Boosting
Section 5: LightGBM (simple)
Section 6: Optuna Tuning (50 trials)
Section 6: LightGBM (tuned)

↓

All models evaluated on Test Set

↓

Compare 4 metrics: Accuracy, F1, ROC-AUC, MCC

Expected Results on Heart Failure Data

Typical Performance:

Model	Accuracy	F1	ROC-AUC	MCC
Random Forest	0.78	0.72	0.81	0.58
Gradient Boosting	0.80	0.74	0.83	0.61
LightGBM (Simple)	0.80	0.73	0.84	0.62
LightGBM (Tuned)	**0.82**	**0.76**	**0.86**	**0.65**

Key Observations:

- Tuning improves **all 4 metrics**
- GB $>$ RF consistently (sequential $>$ parallel)
- LightGBM competitive or better than GB
- No single model dominates all metrics
- Gap between best and worst: $\sim$4-5% ROC-AUC (clinically significant!)

Hyperparameter Tuning: Mistakes to Avoid

Mistake 1: Tuning on test set

- ✗ Leads to overfitting to test data
- ✓ Use cross-validation on training set

Mistake 2: Too few trials

- ✗ 5-10 trials is random luck
- ✓ Use 50+ trials

Mistake 3: Ignoring min_data_in_leaf

- ✗ LightGBM overfits on small data (n=299)
- ✓ Set min_data_in_leaf $\geq$ 20

Mistake 4: Optimizing accuracy only

- ✗ Ignores class imbalance (10% positive)
- ✓ Optimize ROC-AUC, report all 4 metrics

Mistake 5: Different metrics for train/test

- ✗ Confuses signal from noise
- ✓ Use same metric everywhere

Exercise 1: Learning Rate Impact

Task: Compare two learning rates

- **Model A:** learning_rate=0.1 (current)
- **Model B:** learning_rate=0.05 (slower)
- Keep: n_estimators=100, max_depth=3

What to Compare:

- Which model is more accurate?
- Which has better F1-score?
- Which has higher ROC-AUC?

Expected Insight:

- lr=0.1: Faster convergence, might overshoot optimum
- lr=0.05: Slower, more stable, often better on test set
- Small data (n=299) usually favors slower learning

Your Task: Run both models and report all 4 metrics for each.

Exercise 2: Tuning Speed vs Quality

Task: Compare tuning budget impact

- **Version 1:** n_trials=20
  - Time: ∼2 minutes
  - Expected ROC-AUC: 0.83
- **Version 2:** n_trials=50 (current)
  - Time: ∼5 minutes
  - Expected ROC-AUC: 0.86
- **Version 3:** n_trials=100 (if brave)
  - Time: ∼10 minutes
  - Expected ROC-AUC: 0.865

Your Task:

1. Run Optuna with n_trials=20
2. Compare best hyperparameters vs n_trials=50
3. Assess: Is the extra 3 minutes worth 0.03 ROC-AUC gain?

Expected Learning: Diminishing returns in optimization

Exercise 3: Choose Your Own Adventure

Select ONE path:

Option A: Aggressive Tuning (Wider Search)

- Expand ranges: learning_rate 0.001-0.3, num_leaves 31-100
- Explore extreme values
- Best if: You have computational time and like exploration

Option B: Manual Theory-Based Tuning

- Set hyperparameters from learned principles
- learning_rate=0.05, num_leaves=15 (conservative)
- Best if: You want to apply lessons learned

Option C: Ultra-Conservative Model

- Optimize for stability over accuracy
- Very strict regularization (min_data_in_leaf=30)
- Best if: You're concerned about overfitting

Your Task:

1. Choose A, B, or C

Building Intuition: Bias-Variance Tradeoff

Model Complexity $\rightarrow$

High Variance (Overfitting)	**Low Variance (Underfitting)**
Shallow trees	Deep trees
Slow learning rate	Fast learning rate
Small n_estimators	Large n_estimators
High regularization	No regularization

Where models fall on this spectrum:

- **Random Forest:** Deep trees, many parallel
  $\rightarrow$ Low bias, high variance
- **Gradient Boosting:** Shallow trees, sequential
  $\rightarrow$ Medium bias, medium variance
- **LightGBM Tuned:** Leaf-wise optimization
  $\rightarrow$ Low bias, low variance ✓
- **LightGBM Conservative:** Strict rules
  $\rightarrow$ High bias, very low variance

How LightGBM Actually Makes Decisions

Decision at each expansion step:

```
When expanding a tree:

1. Current state: 10 leaves (potential splits)

2. Find the leaf with highest loss reduction:
   - Leaf A: Splitting reduces loss by 0.05
   - Leaf B: Splitting reduces loss by 0.12
      Choose this
   - Leaf C: Splitting reduces loss by 0.03

3. Smart focus on high-error regions:
   - LightGBM: Splits Leaf B first (0.12 > others)
   - Traditional GB: Splits ALL leaves at depth d
                     equally (wastes effort)

4. Result:
   - LightGBM: Fewer leaves, faster training
   - Traditional: More leaves, slower, same/worse
```

Real-World Application: Building a Survival Predictor

Our Dataset:

- **299 heart failure patients**
- **11 clinical measurements** (age, ejection fraction, etc.)
- **Target:** Mortality (10% death rate)
- **Imbalance:** Heavy class imbalance

Algorithm Choice Rationale:

When	Algorithm	Why
Baseline	Random Forest	Easy to understand, fast, good benchmark
Better accuracy	Gradient Boosting	Sequential learning corrects RF errors
Production	LightGBM	Fast, tunable, handles constraints

Real Constraints:

- **Small dataset** (n=299) $\rightarrow$ need strong regularization
- **Imbalanced classes** (10% positive) $\rightarrow$ need multiple metrics
- **Medical domain** $\rightarrow$ need interpretable results

Connecting to Chicco & Jurman (2020)

The Paper's Main Finding:

For imbalanced binary classification, **MCC** *is the most robust metric*
that provides a single quality measure valid for both classes.

How Week 6 Applies This:

- ✓ Use heart failure data (imbalanced: 10% positive)
- ✓ Report all 4 metrics (not accuracy alone)
- ✓ Compare multiple models (RF, GB, LightGBM)
- ✓ Use cross-validation (honest, unbiased estimates)
- ✓ Follow ensemble best practices

The Key Insight:

- Accuracy alone misleads on imbalanced data
- MCC catches weak models that accuracy misses
- Ensemble methods beat single models consistently
- Proper tuning gains: +0.04-0.05 ROC-AUC is clinically meaningful

Key Takeaways

1. **Algorithm Progression**
   - Random Forest: Parallel, simple, good baseline
   - Gradient Boosting: Sequential, better, more complex
   - LightGBM: Optimized, practical, fastest
2. **Why Sequential > Parallel**
   - GB learns from residuals $\rightarrow$ reduces bias + variance
   - RF only reduces variance via averaging
3. **Metrics Matter**
   - Accuracy alone is misleading
   - MCC and ROC-AUC catch hidden problems
   - Report all four, compare all
4. **Hyperparameter Tuning**
   - Optuna: Smart search saves time
   - Cross-validation: Essential for honest estimates
   - Regularization: Prevent overfitting on small data
5. **Production Readiness**
   - LightGBM is 10-20$\times$ faster

What's Next (Weeks 7+)

You now know:

- ✓ Compare different algorithms systematically
- ✓ Evaluate with multiple metrics
- ✓ Tune hyperparameters automatically
- ✓ Build ensemble models

Coming next:

- Advanced feature engineering
- Handling class imbalance (SMOTE, class_weight)
- Model interpretation (SHAP values, feature importance)
- Production deployment (model saving, serving)
- Cross-validation strategies for small data

The big picture:

Systematic, data-driven approach beats guessing every time.

Resources for Deeper Learning

Gradient Boosting Intuition

- StatQuest: Gradient Boost (YouTube)
- StatQuest: XGBoost (YouTube playlist)

LightGBM Mastery

- LightGBM Official Documentation
- Parameter Tuning Guide

Hyperparameter Optimization

- Optuna Official Documentation
- Hyperparameter Optimization Strategies Tutorial

Evaluation Best Practices

- Chicco & Jurman (2020): MCC Advantages
- Imbalanced-Learn Library

Live Examples

- Kaggle competitions using LightGBM
- GitHub: LightGBM competition winners

Q&A / Open Discussion

Q: Why not just use the simplest model?

- A: Tuning gives +0.03-0.05 ROC-AUC on this dataset. In medical contexts, this can save lives.

Q: Is LightGBM always better?

- A: Not always. On very small data (¡100 samples) or with outliers, simpler models work better.

Q: How many trials do I need?

- A: 50 is good for this problem. More trials rarely help beyond n=100 (diminishing returns).

Q: What if my tuned model is worse on test data?

- A: Use CV-based tuning on training set. Report test-set results separately.

Q: Should I report accuracy or ROC-AUC?

- A: Both. Report all four metrics. ROC-AUC more reliable for imbalanced data.

The Big Picture: From Week 1 to Week 6

Progression:

Week 1-2	Data loading + cleaning
Week 3-4	Exploratory analysis + feature engineering
Week 5	Baseline modeling (Random Forest)
Week 6	**Systematic improvement (GB + LightGBM)**
Week 7+	Advanced techniques + production

What this progression teaches:

1. **Start simple** (RF: 5 hyperparameters)
2. **Improve systematically** (GB: understand residuals)
3. **Optimize carefully** (LightGBM: 7 hyperparameters + Optuna)
4. **Evaluate honestly** (4 metrics + cross-validation)
5. **Scale confidently** (ready for production)

Good machine learning is iterative, not magical.

Simple baselines $\rightarrow$ Systematic improvements $\rightarrow$ Production systems

Thank You

Week 6: From Random Forest to LightGBM

Questions? Comments? Observations?

Remember: The best model is the one you understand and can explain to others.